

本次修改说明：
1. 增加 LTC 和 BCH 多签：
 • LTC 和 BCH 多签流程同 BTC。

SDK 支持 Hcash multisig 说明：

- 新增 JUB_CreateMultiSigContextHC 接口入参 ContextCfgMultisigHC；
- 接口 JUB_SignTransactionHC 增加返回值 JUBR_MULTISIG_OK(0x0000000B) 用以判断 multisig 是否完成，若未完成（即签名个数未达到 m-n 中 m 的值），则继续签；若返回 JUBR_OK，则交易完成，可以公布交易。

• Hcash multisig 签名流程 说明：

1. 第一个人签名（参考 demo 中的 transaction_test_hcash()）：
 - JUB_CreateMultiSigContextHC (ContextCfgMultisigHC)
 - verify pin
 - JUB_SetUnitBTC
 - JUB_SignTransactionHC 若返回 JUBR_MULTISIG_OK，则继续第 2 个流程；若返回 JUBR_OK, multisig 交易完成。
2. 非第一个人签名：
同第一个人签名流程。

SDK 支持 BTC multisig 说明：

- 修改 JUB_CreateContextBTC 接口入参 ContextConfigBTC，创建用于 multisig 的操作时需要使用 ContextCfgMultisigBTC；
- 新增 JUB_ParseTransactionBTC 接口，用于解析未完成的 multisig 的交易raw；
- 接口 JUB_SignTransactionBTC 增加返回值 JUBR_MULTISIG_OK(0x0000000B) 用以判断 multisig 是否完成，若未完成（即签名个数未达到 m-n 中 m 的值），则继续签；若返回 JUBR_OK，则交易完成，可以公布交易。

• BTC multisig 签名流程 说明：

1. 第一个人签名（参考 demo 中的 transaction_test()/transaction_test_hcash()）：
 - JUB_CreateContextBTC (ContextCfgMultisigBTC) /JUB_CreateMultiSigContextHC (ContextCfgMultisigHC)
 - verify pin
 - JUB_SetUnitBTC
 - JUB_SignTransactionBTC/JUB_SignTransactionHC
2. 非第一个人签名（参考 demo 中的 transactionMultisig_test()/transaction_test_hcash()）：
 - JUB_CreateContextBTC (ContextCfgMultisigBTC) /JUB_CreateMultiSigContextHC (ContextCfgMultisigHC)
 - verify pin
 - JUB_SetUnitBTC
 - JUB_ParseTransactionBTC 解析 hexIncTx，并将 hexIncTx 缓存到当前 ContextCfgMultisigBTC 中
 - 完善 JUB_ParseTransactionBTC 所返回的 vInputs 和 vOutputs 信息（参考 transactionMultisig_test() 中 vInputs 和 vOutputs 的 update 流程）
 - JUB_SignTransactionBTC
 - JUB_SignTransactionBTC 若返回 JUBR_MULTISIG_OK，则继续第 2 个流程；若返回 JUBR_OK, multisig 交易完成。

• 多签参数说明：

- 创建 context

```
//typedef struct stContextCfg {  
//    JUB_CHAR_PTR    main_path;  
//} CONTEXT_CONFIG;  
@interface ContextConfig : NSObject  
@property (atomic, copy ) NSString* mainPath;    // 多签 mainPath 为 "m/45"  
@end  
  
//typedef struct stContextCfgBTC : stContextCfg {  
//    JUB_ENUM_COIN_TYPE BTC    coinType; //= { JUB_ENUM_COIN_TYPE BTC::COINBTC };  
//    JUB_BTC_TRANS_TYPE    transType;  
//} CONTEXT_CONFIG_BTC;  
@interface ContextConfigBTC : ContextConfig  
@property (atomic, assign) JUB_NS_ENUM_COIN_TYPE BTC coinType; // NS_COINBTC(0x00)  
@property (atomic assign) JUB_NS_BTC_TRANS_TYPE transType;    // NS_P2SH_MULTISIG(0x02)  
@end
```

```
//typedef struct stContextCfgMultisigBTC : stContextCfgBTC {  
//    unsigned long m;  
//    unsigned long n;  
//    std::vector<std::string> vCosignerMainXpub;  
//} CONTEXT_CONFIG_MULTISIG_BTC;  
@interface ContextCfgMultisigBTC : ContextConfigBTC  
@property (atomic, assign) NSUInteger m;            // 至少需要的合作者的个数  
@property (atomic, assign) NSUInteger n;            // 合作签名者的个数  
@property (atomic, strong) NSMutableArray* vCosigner    // 合作签名者的 mainXpub (m/45')  
@end
```

举例：A、B、C 三个合作签名者，要进行 2-3 的多签，则 m=2, n=3；在 A 进行签名时，vCosigner 给 B 和 C 的 mainXpub；在 B 进行签名时，vCosigner 给 A 和 C 的 mainXpub~~~

- 第一次签名的 input/output 准备
 - 第一个签名者进行签名时，他既是第一个签名的人，也是组多签交易的人，其 input/output 参考以下 JSON 所给的信息：

```
"inputs" : [  
  {  
    "multisig" : true,  
    "preHash" : "210a08785ed5590795869d37458337514a81d022bdfd5115bf55e52ed106400a3",  
    "preIndex" : 1,  
    "amount" : 17900,  
    "bip32_path" : {  
      "change" : false,  
      "addressIndex" : 0  
    }  
  },  
],  
"outputs" : [  
  {  
    "multisig" : true,  
    "address" : "3KwVRo3wycw2jpWnrW62VzeztjypHWGiWz",  
    "amount" : 19000,  
    "change_address" : true,  
    "bip32_path" : {  
      "change" : true,  
      "addressIndex" : 0  
    }  
  }  
],
```

其中，input.type 和 output.type 的值，参考 JUB_NS_SCRIPT_BTC_TYPE

```
//typedef enum {  
//    P2PKH    = 0x00,  
//    RETURN0  = 0x01,  
//    P2SH_MULTISIG = 0x02  
//} SCRIPT_BTC_TYPE;  
typedef NS_ENUM(NSInteger, JUB_NS_SCRIPT_BTC_TYPE) {  
    NS_P2PKH = 0x00,  
    NS_RETURN0 = 0x01,  
    NS_P2SH_MULTISIG = 0x02,  
    NS_SCRIPT_BTC_TYPE_NS  
};
```

如果 input 是 P2PKH 地址，则该 input.type = 0x00.

- 第 N 次签名的 input/output 准备
在第一个完成签名之后，JUB_SignTransactionBTC 返回 JUBR_MULTISIG_OK，即获得所谓“已签名未完成交易”，需要进行后续签名。”已签名未完成交易“需要使用 JUB_ParseTransactionBTC 进行解析。然后，将其返回的 inputs 和 outputs 进行信息补充，以便完成接下来的签名。信息补充参考下图 黄框 所示：

```
"inputs" : [  
  {  
    "multisig" : true,  
    "preHash" : "210a08785ed5590795869d37458337514a81d022bdfd5115bf55e52ed106400a3",  
    "preIndex" : 1,  
    "amount" : 17900,  
    "bip32_path" : {  
      "change" : false,  
      "addressIndex" : 0  
    }  
  },  
],  
"outputs" : [  
  {  
    "multisig" : true,  
    "address" : "3KwVRo3wycw2jpWnrW62VzeztjypHWGiWz",  
    "amount" : 19000,  
    "change_address" : true,  
    "bip32_path" : {  
      "change" : true,  
      "addressIndex" : 0  
    }  
  }  
],
```

• 不支持混签类型的交易

